

SEO PACKAGE

1. SEO Title

Reverse Degree of a String (LeetCode 3498) — Python Solution Explained with the ord() Trick

2. SEO Subtitle

A step-by-step breakdown of LeetCode 3498 that shows why reversing the string is a trap, and how a single `ord()` call replaces it entirely.

3. Featured Quote

"A lot of these easy problems invent their own numbering — reversed, offset by one, whatever. Read the definition twice before you write any code."

4. Meta Description

Solve LeetCode 3498, Reverse Degree of a String, in Python with $O(n)$ time and $O(1)$ space. Learn the `ord()` trick and avoid the reversed-alphabet trap.

5. URL Slug

leetcode-3498-reverse-degree-of-a-string-python

6. Keywords

LeetCode 3498, Reverse Degree of a String, LeetCode Python solution, LeetCode daily challenge, Python `ord()` function, Python `enumerate`, coding interview prep, string manipulation Python, algorithm intuition, $O(n)$ time complexity, $O(1)$ space complexity, LeetCode easy problems, Python interview questions, alphabet position problem, reversed alphabet value, data structures and algorithms, DSA Python, string traversal algorithm, LeetCode explained, competitive programming Python

7. Tags

LeetCode, Python, Coding Interview, Algorithms, Data Structures, DSA, LeetCode Daily, Software Engineering, String Problems, Interview Prep, Python Tutorial, ord Function, Easy LeetCode

MAIN ARTICLE

Introduction

Easy-rated LeetCode problems have a reputation for being throwaway warmups, but the ones that stick around in interview loops are usually the ones with a small twist buried in the definition. Reverse Degree of a String is exactly that kind of problem. On the surface it looks like a basic loop-and-sum exercise. Underneath, it's testing something much more valuable than syntax: can you read a spec carefully enough to catch a numbering convention that runs backward from what you'd naturally assume?

This matters beyond the problem itself. Interviewers use problems like this to see whether you verify your understanding against a worked example before committing to code, or whether you charge ahead on your first guess. Getting burned once by "reverse" meaning something other than what you expected is a cheap lesson to learn here, and a very expensive one to learn live in an interview.

Problem Overview

You're given a string made up entirely of lowercase English letters. Every letter in the alphabet has a "reversed value" — instead of `a = 1, b = 2, ..., z = 26`, the values run the other direction: `a = 26, b = 25, ..., z = 1`.

For each character in the string, take its reversed alphabet value and multiply it by its position in the string, counting positions from 1 (not 0). Sum all of these products together. That sum is the string's "reverse degree."

The string itself is never reversed. Only the alphabet-to-number mapping is reversed. That distinction is the entire problem.

Example

Take the string `"abc"`.

Character Position (1-indexed) Reversed alphabet value Product

a	1	26	26
b	2	25	50
c	3	24	72

Sum: $26 + 50 + 72 = 148$.

So "abc" has a reverse degree of 148. Every product comes from the same rule — reversed letter value times normal position — and the total is just the running sum of those three numbers.

Intuition

The natural first instinct when you see the word "reverse" in a string problem is to reverse the string, or reverse the order you iterate through it. That instinct is wrong here, and it's wrong in a way that's easy to miss because the math still produces a plausible-looking number.

A more common first mistake is assuming $a = 1, b = 2, c = 3$ — normal alphabet order — because that's the mapping most people reach for by default. Running that on "abc" gives $1*1 + 2*2 + 3*3 = 1 + 4 + 9 = 14$. That's not 148, so something in the assumption is wrong.

Rereading the problem statement clarifies it: the position in the *alphabet* is reversed, not the string or the loop direction. **a** is worth 26 because it's 26th from the end of the alphabet; **z** is worth 1 because it's 1st from the end. The string is still processed left to right, in its normal order, with normal 1-indexed positions.

This is the core skill the problem is testing: distinguish "reverse the alphabet mapping" from "reverse the string" or "reverse the index." Once you fix the mapping, everything else is a one-pass sum you've written a hundred times before.

Brute Force Solution

Idea: For each character, compute its reversed alphabet value with an explicit lookup (a dictionary or a 26-entry list built from 'a' to 'z'), multiply by 1-indexed position, and accumulate.

Algorithm: 1. Build a mapping of each letter to its reversed value (either precomputed or via a formula). 2. Loop through the string with an index. 3. Multiply and accumulate.

Advantages: Very easy to reason about; the mapping is explicit and visible.

Disadvantages: Building an explicit 26-entry table is unnecessary work when the value can be computed directly from each letter's ASCII code.

Complexity: $O(n)$ time, $O(1)$ extra space (the alphabet table is a constant size regardless of input length) — so in practice this isn't meaningfully slower than the optimal version, just more code for the same result.

Optimal Solution

Skip the explicit lookup table. Python's `ord()` function already gives you a letter's position in the normal alphabet as an ASCII offset: `ord(char) - ord('a')` returns `0` for `'a'`, `1` for `'b'`, up through `25` for `'z'`.

To flip that into the reversed value, just invert it: `26 - (ord(char) - ord('a'))`. For `'a'`, that's `26 - 0 = 26`. For `'z'`, that's `26 - 25 = 1`. That single expression replaces the entire lookup table.

Then use `enumerate()` to walk the string and get both the index and the character in one pass. Since `enumerate` starts counting at 0 but the problem wants 1-indexed positions, add 1 to the index before multiplying.

```
position(0-indexed) → +1 → actual position used in the formula
character           → ord() flip → reversed alphabet value
                    → multiply → add to running total
```

Python Code

```
def reverse_degree(s: str) -> int:
    total = 0
    for index, char in enumerate(s):
        reversed_value = 26 - (ord(char) - ord('a'))
        position = index + 1
        total += reversed_value * position
    return total
```

Code Walkthrough

- `total = 0` — the running sum, returned at the end.
- `enumerate(s)` — yields `(index, char)` pairs so there's no manual counter to maintain.
- `ord(char) - ord('a')` — converts the letter into its normal 0-based alphabet position (`a` → 0, `z` → 25).
- `26 - (...)` — flips that into the reversed value (`a` → 26, `z` → 1).

- `position = index + 1` — shifts `enumerate`'s 0-based index into the problem's 1-based position.
- `total += reversed_value * position` — accumulates each character's contribution.
- `return total` — the final reverse degree.

Dry Run

String: `"abc"`

index	char	ord(char)-ord('a')	reversed_value	position	product	total
0	a	0	26	1	26	26
1	b	1	25	2	50	76
2	c	2	24	3	72	148

Final answer: `148`, matching the example.

Complexity Analysis

Time: $O(n)$, where n is the length of the string. Each character is visited exactly once, and the work per character (an `ord()` call, a subtraction, a multiplication) is constant time.

Space: $O(1)$. Only a running total, an index, and a character are held at any point — none of that scales with input size.

These are correct because there's no nested loop, no auxiliary data structure that grows with input, and no recursion.

Alternative Solutions

You could precompute a dictionary mapping each of the 26 letters to its reversed value up front (`{chr(97+i): 26-i for i in range(26)}`) and do a dictionary lookup instead of the `ord()` arithmetic inside the loop. This is functionally identical — still $O(n)$ time, still $O(1)$ space (the dictionary is a fixed 26 entries) — but it's more code to build a table `ord()` already computes for you inline. Fine in interviews if you want to make the mapping visually explicit, but not necessary.

Edge Cases

- **Single-letter string:** e.g., "z" — position is 1, so the answer is just z's reversed value, which is 1. No special-casing needed; the loop handles it naturally.
- **All same letter:** e.g., "aaa" — reversed value stays 26 for every character, only the position changes, so the answer is $26 * (1+2+3) = 156$.
- **Maximum length (1000 characters):** still resolves in a single linear pass; no performance concern.
- **String of z's:** each contributes only 1 times its position, so the total will be much smaller than the same-length string of a's.
- **Empty string:** not part of the problem's constraints (length is at least 1), but the code would correctly return 0 if it were.

Common Mistakes

1. Using normal alphabet order (a=1, b=2, ... z=26) instead of the reversed order the problem actually specifies.
2. Reversing the string itself instead of reversing the alphabet-to-number mapping.
3. Forgetting to shift enumerate's 0-based index by adding 1, which throws off every product.
4. Reversing the index weighting (processing positions back-to-front) rather than the letter values.
5. Building a lookup table with an off-by-one error, e.g. treating a as 25 instead of 26.
6. Not verifying the formula against the given example before writing code, and only catching the mistake after submission fails.

Interview Questions

- How would you handle uppercase letters or mixed case if the constraints changed?
- Can you compute the reverse degree without a loop, using a formula or vectorized approach?
- What if positions were 0-indexed instead of 1-indexed — how would the code change?
- How would you extend this to a non-English alphabet or a different base value than 26?

Similar Problems

- **LeetCode 1: Two Sum** — another problem where a single pass with a running structure (hash map instead of a running total) solves it in $O(n)$.
- **LeetCode 806: Number of Lines To Write String** — also maps characters to fixed values via a lookup and accumulates a running total.
- **LeetCode 1417: Reformat The String** — string traversal with position-based logic driving the output.

- **LeetCode 2085: Count Common Words With One Occurrence** — reinforces reading problem statements carefully since the "obvious" interpretation is often wrong.

Key Takeaways

The algorithm here is trivial once you have the right mapping — a single pass, multiply, and sum. The actual skill being tested is reading comprehension: catching that "reverse" applies to the alphabet values, not the string or the iteration order. `ord()` replaces an entire lookup table with one line of arithmetic, and `enumerate()` removes the need to track a manual counter. Verify your formula against the given example before writing any code — it would have caught the wrong-mapping mistake in seconds instead of after a failed submission.

Watch the Video

For the full walkthrough, including the trace of the example step by step and two quick quiz rounds to test your understanding, watch the video here: {LINK}

About the Series

This breakdown is part of Fun with Learning Technology, a series that works through LeetCode's daily challenge with a focus on the reasoning behind each solution — not just the final code, but how an experienced engineer catches the trap, tests the assumption, and arrives at the fix.

Call To Action

If this helped the problem click, give the video a like on YouTube and subscribe for the rest of the daily LeetCode series. For deeper write-ups and more Python breakdowns, subscribe to the Substack. Drop a comment with your own approach or a question about the edge cases, and share this with anyone prepping for interviews.

SOCIAL MEDIA

LinkedIn Post

LeetCode 3498, Reverse Degree of a String, looks like a basic loop problem until you actually read the spec closely.

The setup: each letter gets a value based on its position counting backward through the alphabet ($a = 26, b = 25, \dots, z = 1$), multiplied by its normal position in the string (counting from 1), summed across all characters.

My first instinct was to use normal alphabet order — $a=1, b=2, c=3$. For "abc" that gives 14. The expected answer is 148. The gap wasn't a bug in the loop; it was a wrong assumption about which direction "reverse" applied to. It's not the string that's reversed, and it's not the iteration order — it's the alphabet-to-number mapping itself.

Once that clicks, the solution is one pass: `26 - (ord(char) - ord('a'))` gives the reversed value directly, no lookup table needed, and `enumerate()` handles position tracking. $O(n)$ time, $O(1)$ space.

The bigger lesson isn't about this specific problem. It's that "easy" problems often invent their own numbering conventions — reversed, offset by one, zero-indexed vs one-indexed — and the fastest way to lose time isn't a hard algorithm, it's writing code against an assumption you never checked with the example.

Full breakdown and code walkthrough: {LINK}

Python #LeetCode #CodingInterview

#Algorithms #SoftwareEngineering

Twitter/X Thread

1/ LeetCode 3498: Reverse Degree of a String. Looks trivial. Isn't — the trap is in one word. Thread 🧵

2/ The rule: each letter gets a value based on its position counting BACKWARD through the alphabet. $a=26, b=25, \dots, z=1$. Multiply by the letter's normal position in the string (1-indexed). Sum it all.

3/ First instinct: $a=1, b=2, c=3$, normal order. For "abc": $11 + 22 + 3*3 = 14$.

4/ Expected answer for "abc": 148. Not close. Something's fundamentally wrong with the assumption, not the arithmetic.

5/ Reread the spec: "reversed alphabet position." The word "reverse" applies to the alphabet mapping — not the string, not the loop direction.

6/ Correct mapping: a=26, b=25, c=24. Redo it: $261 + 252 + 24 \cdot 3 = 26 + 50 + 72 = 148$. 

7/ The code needs exactly one trick: `26 - (ord(char) - ord('a'))` turns any lowercase letter into its reversed value. No lookup table required.

8/ Pair that with `enumerate()` for position tracking, add 1 since enumerate starts at 0, multiply, accumulate.

9/ Full solution is 5 lines. $O(n)$ time, $O(1)$ space. The algorithm was never the hard part.

10/ The real lesson: easy problems love inventing their own numbering conventions. Read the definition twice, hand-check one example, then write code. Full breakdown: {LINK}

Facebook Post

Ever hit a coding problem where your first answer is confidently, completely wrong — and it takes rereading the same sentence three times to see why?

That happened with LeetCode's Reverse Degree of a String. The word "reverse" makes you think you should flip the string, or process it backward. Nope. What's actually reversed is how letters get scored: a is worth 26, z is worth 1, and the string stays in its normal order the whole time.

Once that clicked, the fix was a single line of Python using `ord()` — no lookup tables, no extra complexity. Full walkthrough, code, and a couple of quiz questions to test yourself: {LINK}

Reddit Summary

Worked through LeetCode 3498 (Reverse Degree of a String) — an easy-rated problem where the actual difficulty is in the wording, not the algorithm.

The rule: each character's value is based on its position counting backward through the alphabet (a=26 ... z=1), multiplied by its 1-indexed position in the string, summed up.

The natural mistake is assuming normal alphabet order (a=1, b=2...) instead of reversed. Running the example ("abc") with the wrong mapping gives 14; the correct mapping gives 148. The word "reverse" refers to the alphabet-to-value mapping, not the string itself or the iteration direction.

Once you have the right mapping, the Python solution is short:

```
def reverse_degree(s):
    total = 0
    for i, c in enumerate(s):
```

```
total += (26 - (ord(c) - ord('a')) * (i + 1)
return total
```

$O(n)$ time, $O(1)$ space. Wrote up the full reasoning, dry run, and edge cases if anyone wants the longer version.

GITHUB README

Reverse Degree of a String — LeetCode 3498 (Python)

Problem

Given a string `s` consisting only of lowercase English letters, compute its **reverse degree**:

- Each letter has a value based on its position counting *backward* through the alphabet: `a = 26`, `b = 25`, ..., `z = 1`.
- For every character, multiply its reversed alphabet value by its 1-indexed position in the string.
- Sum all the products to get the reverse degree.

Example: `s = "abc"` → `26*1 + 25*2 + 24*3 = 148`

Intuition

The trap is assuming normal alphabet order (`a=1`, `b=2`, ...). That gives the wrong answer. The word "reverse" refers to the alphabet-to-value mapping, not the string order or iteration direction. The string is processed left to right as usual; only the letter values are flipped.

Approach

- Walk the string once with `enumerate()` to get index and character together.
- Convert each character to its reversed alphabet value: `26 - (ord(char) - ord('a'))`.
- Multiply by the 1-indexed position (`index + 1`).
- Accumulate into a running total.

Python Solution

```
def reverse_degree(s: str) -> int:
    total = 0
    for index, char in enumerate(s):
        reversed_value = 26 - (ord(char) - ord('a'))
        position = index + 1
        total += reversed_value * position
    return total
```

Complexity

- **Time:** $O(n)$ — single pass over the string.
- **Space:** $O(1)$ — only a running total and loop variables.

Video

Full walkthrough with dry run and quiz questions: {LINK}

Article

Full write-up with brute force comparison, edge cases, and interview follow-ups: {LINK}

Quick Review

Pattern: 'reverse degree' via char position weighting — trigger?

When score depends on $(26 - \text{letter_rank})$ times position, think weighted-sum-in-one-pass, no need for extra structures.

Time complexity of computing Reverse Degree?

$O(n)$ — one linear scan over the string computing weight and summing.

Space complexity of the straightforward solution?

$O(1)$ extra space — just running total and index counters, no arrays needed.

Common bug when mapping 'a'→26 down to 'z'→1?

Off-by-one from `ord(c) - ord('a')`: forgetting to invert ($26 - (\text{ord}(c) - \text{ord}('a'))$) gives 'a' the wrong weight.

Bug from using 0-indexed position directly in the sum?

Position must be 1-indexed ($i+1$) when multiplying by weight, else the first character always contributes zero.

Interview follow-up: how to extend to 'sum over every substring'?

Precompute prefix sums of weighted values so each substring's contribution is $O(1)$ via `prefix[j] - prefix[i]`, avoiding $O(n^2)$.